

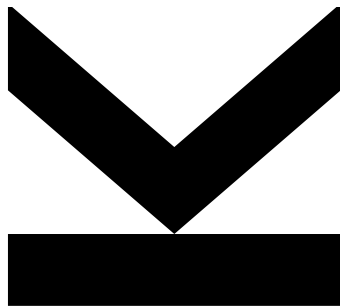
Submitted by
Stefan Fischereider

Submitted at
**LIT Cyber-Physical Sys-
tems Lab**

Supervisor
**Univ.-Prof. DI Dr. Alois
Zoitl**

May 2021

Development of a Next Generation SFC Viewer Based on Web Technolo- gies



Master Thesis
to obtain the academic degree of
Diplom-Ingenieur
in the Master's Program
Elektronik und Informationstechnik

Affidavit

I hereby declare under oath that the submitted Master's Thesis has been written solely by me without any third-party assistance, information other than provided sources or aids have not been used and those used have been fully documented. Sources for literal, paraphrased and cited quotes have been accurately credited.

The submitted document here present is identical to the electronically submitted text document.

Linz, May 2021

A handwritten signature in blue ink, reading "Fischereder", enclosed within a thin rectangular border.

.....
Stefan Fischereder

Acknowledgements

First of all I want to thank Univ.-Prof. DI Dr. Alois Zoitl for handing me the opportunity to write this thesis at the LIT Cyber-Physical Systems Lab. Also I want to express my gratitude for his continuous support and advisory on writing the thesis, often times very spontaneous as is required by these challenging times.

Additionally I want to thank everyone at KEBA AG who supported me while working on the project, and for providing advice whenever needed.

Last but not least, I want to express my deepest gratitude to my family and friends for their unconditional support throughout all the years of study.

*Der Himmel ist blau
Und der Rest deines Lebens liegt vor dir
Vielleicht wäre es schlau
Dich ein letztes Mal um zusehen
Du weißt nicht genau warum
Aber irgendwie packt Dich die Neugier
Der Himmel ist blau
Und der Rest deines Lebens wird schön, yeah*

Die Ärzte, Punk-Rock Band, Berlin

Abstract

This thesis shows a possible way of displaying an automatically generated graphical Sequential Function Chart with web-technologies as a technological base. The motivation behind developing the SFC Viewer is the need for a future-oriented technology-base and the urge for constant renewal of the tool-chain for developing projects for the IEC 61131-3 standard. First, the current state-of-the-art will be analysed. We take a look at the IEC 61131-3 standard, the editor base Visual Studio Code, the concepts of the Language Server Protocol and theoretical basics for drawing graphs. In the next step we will analyse the system more precisely in performing a use case analysis, which will lead to the requirements for the SFC Viewer, and will lay the grounds for creating a basic concept as well as a general software architecture. After the architecture is established, the drawing framework will be selected, its own architecture explained and the inner workings of the SFC Viewer will be covered in more detail. The working prototype is evaluated against existing examples, it is capable of showing basic SFCs and the identified use cases are covered by implementing their respective functional requirements. Finally, user feedback is incorporated in the use case model, which finally leads to a refinement of the requirements. The next steps for the SFC viewer are obviously continuing to enhance the overall usability which will be triggered by further user testing, and ultimately the extension to a full-fledged editor, which can be used in a standalone way.

Kurzfassung

Diese Arbeit zeigt einen möglichen Weg zur Darstellung eines automatisch generierten grafischen Sequential Function Chart mit Web-Technologien als technologische Basis. Die Motivation für die Entwicklung des SFC-Viewers ist der Bedarf an einer zukunftsorientierten Technologiebasis und der Drang zur ständigen Erneuerung der Werkzeugkette zur Entwicklung von Projekten für die IEC 61131-3 Norm. Zunächst wird der aktuelle Stand der Technik analysiert. Wir werfen einen Blick auf die IEC 61131-3-Norm, die Editor-Basis Visual Studio Code, die Konzepte des Language Server Protocol und theoretische Grundlagen zum Zeichnen von Graphen. Im nächsten Schritt werden wir das System genauer analysieren, indem wir eine Use-Case-Analyse durchführen, die zu den Anforderungen für den SFC-Viewer führt und die Grundlage für die Erstellung eines Grundkonzeptes sowie einer allgemeinen Software-Architektur bildet. Nachdem die Architektur festgelegt ist, wird das Zeichen-Framework ausgewählt, dessen Architektur erläutert und das Innenleben des SFC-Viewers näher beleuchtet. Der funktionierende Prototyp wird anhand von existierenden Beispielen evaluiert, er ist in der Lage, grundlegende SFCs darzustellen und die identifizierten Use-Cases werden durch die Umsetzung der jeweiligen funktionalen Anforderungen abgedeckt. Schließlich wird das Feedback der Benutzer in das Use-Case-Modell eingearbeitet, was schließlich zu einer Verfeinerung der Anforderungen führt. Die nächsten Schritte für den SFC-Viewer sind natürlich die weitere Verbesserung der Benutzerfreundlichkeit, die durch weitere Benutzertests angestoßen wird, und schließlich die Erweiterung zu einem vollwertigen Editor, der eigenständig genutzt werden kann.

Contents

1	Introduction	1
1.1	Scope	2
1.2	Approach	2
1.3	Guideline Through the Work	2
2	State of the Art	3
2.1	IEC 61131-3	3
2.1.1	Sequential Function Charts	4
2.1.2	SFC examples	5
2.2	Visual Studio Code	6
2.2.1	VSCoDe's Extension API	6
2.3	Language Server Protocol	7
2.3.1	Textual Language Server Protocol	8
2.3.2	Graphical Language Server Protocol and Intermediate Representation Format	8
2.4	Graph Drawing	9
2.4.1	Theoretical Background	9
2.4.2	Layered Algorithm	9
2.4.3	Eclipse Layout Kernel	11
2.5	Summary	11
3	Concept and Analysis	12
3.1	Use Case Analysis	12
3.1.1	System Context and Actors	12
3.1.2	System Use-Cases	13
3.1.3	Redundant Steps	16
3.1.4	Activity Diagrams	17
3.1.5	Requirements	20
3.2	Concept and Architecture	22
3.2.1	Concept	22
3.2.2	Architecture	23

4	Implementation	25
4.1	Comparison of drawing frameworks	25
4.1.1	mxGraph	25
4.1.2	Cytoscape.js	25
4.1.3	Sprotty	26
4.1.4	Selection	26
4.2	Intermediate Representation Format	27
4.3	irf-to-elk Bridge	29
4.4	Sprotty Architecture	31
4.4.1	Introduction	31
4.4.2	Mapping Intermediate Representation Format-Eclipse Layout Kernel-Sprotty	32
4.4.3	Extension integration	35
4.5	Realization	36
4.6	Summary	36
5	Evaluation	37
5.1	Elevator Cabin Control	37
5.2	Elevator Ride Control	38
5.3	Layout of Large SFCs	39
5.4	User Feedback	41
5.4.1	General Remarks	41
5.4.2	Use Case Adaptations and Additional Requirements	41
5.5	Summary	43
6	Conclusion and Outlook	44

Chapter 1

Introduction

IEC 61131-3 (IEC) [5] is the most commonly used programming standard when it comes to realizing projects for industrial automation. It has been released in 1992, thus the projecting and programming tools originally developed for this standard have aged quite a bit. Industrial automation tends to be a very static field when it comes to renewal of its tools, yet, at some point in time, the need for a technology upgrade is getting necessary. The demand of costumers to use state-of-the-art developing standards, such as for example code version control systems like *git* make the developments in this area inevitable.

Since the development of Integrated Development Environments (IDEs) is a highly complex undertaking, it becomes a common interest to lower the complexity of such a task and split it into several sub-projects. This has triggered the community driven development of a so-called Language Server Protocol (LSP). In short the LSP is a communication protocol between a client editor, which wants to get language specific information from a language server. With this approach, only the language server has to be swapped for any other language to be supported by the editor. The developments so far only cover text-based programming languages, yet some research is done in extending this idea to graphical languages as well.

IEC 61131-3 (IEC) contains five programming languages, one of them has both a graphical and textual representation, the Sequential Function Chart (SFC). SFCs are used for top-level organization of processes in an industrial automation context. They consist of Steps, Transitions between those Steps and Actions, which are the executed parts inside a Step. Up until now, the graphical part of the SFC was stored as binary information in the source code file itself, which makes it very difficult to keep track of changes with a version control system.

To solve this issue, an automatic layout of the SFC conforming to the standard should be performed, only then no graphical information needs to be stored anywhere in a file. In order to get consistent results layouting those Graphs, the Eclipse Layout Kernel (ELK) is used. It is an open-source project which provides a wide range of pre-packed layout algorithms of which the layered algorithm is the best fit for flow-chart like graphs, which the SFCs are.

Visual Studio Code (VSCode), Microsoft's editor with full LSP support, is chosen as a base for development. Its extension Application Programming Interface (API) enables users to extend its functionality conveniently, and its browser based framework allows developers to use a wide range of web technologies as well as ELKs JavaScript compiled library.

1.1 Scope

The scope of this thesis is to automatically generate and layout the graphical representation of any given valid Sequential Function Chart (SFC). In order to find the requirements of the system a use case analysis has to be performed. After that the system should be able to layout the graphs deterministically without storing any additional information except the given source code itself. In addition to that, the graphs should look as similar as possible to the representation defined by the IEC 61131-3 (IEC) standard. Another important goal of this work is the definition of an architecture which has the possibility to be extended to a full-fledged graphical editor.

1.2 Approach

A Unified Modeling Language (UML) Use-Case-Analysis based on the methods of [9] will be performed in order to find the systems context and its use cases. The gathered information of the use cases will be used to define the requirements for the project. Several technology options will be tested, in order to find the most suitable framework for the given task. After that a prototype which meets the requirements will be developed and tested.

1.3 Guideline Through the Work

In Chapter 2 the current state-of-the-art is analysed. Additionally short introductions to the used technologies and standards are presented. In Chapter 3 the UML Use-Case-Analysis is performed and the basic ideas for the concept and the architecture of the project are outlined. In the following Chapter 4 the technology comparison is performed and the chosen technology's prototype realization is presented. After that Chapter 5 shows first results of the developed prototype. An analysis of the Use-Cases established in Chapter 3 is performed as well. Finally, a summary of the results is presented in Chapter 6. Additionally, the next steps and an outlook for the project are provided.

Chapter 2

State of the Art

In this chapter we will cover the standards as well as the used technologies and theoretical prerequisites which will be essential for understanding the main ideas of this thesis. First we will take a look at the IEC 61131-3 (IEC) standard, in which we will cover the SFC language in more detail. After that we need to take a look at the editor we used as a base for the project. Visual Studio Code (VSCode) incorporates the ideas of the Language Server Protocol (LSP), which is mainly a protocol for textual languages but can be extended for graphical languages as well. Finally we will examine the basics for graph drawing theory, and especially the layered algorithm approach. Subsequently, the Eclipse Layout Kernel (ELK) is presented as well.

2.1 IEC 61131-3

A Programmable Logic Controller (PLC) is the most frequently used implementation platform for control algorithms in industrial applications. As vendors of PLCs developed their systems, they used different operating systems, runtimes and even programming languages. In order to keep interoperability between systems the IEC 61131 standard was established. Part 3 of this standard IEC 61131-3 (IEC) [5], published in 1992, defines five basic programming languages, in order to abstract the vendor specific specialities: two text-based languages, Instruction List (IL) and Structured Text (ST), two graphical ones, Function Block Diagram (FBD) and Ladder Diagram (LD), and a higher-level language for structuring purposes, which is available in both graphical and textual form, the Sequential Function Charts (SFCs) [11].

Generally, PLC projects are divided into *configurations*, which can hold one or more *resources*. These resources contain *programs* under the control of zero or more *tasks* [5]. Generally the programs are called Program Organization Units (POUs), and are the parts in which variables are defined and one or more of the aforementioned programming languages are used. Figure 2.1 shows this structure.

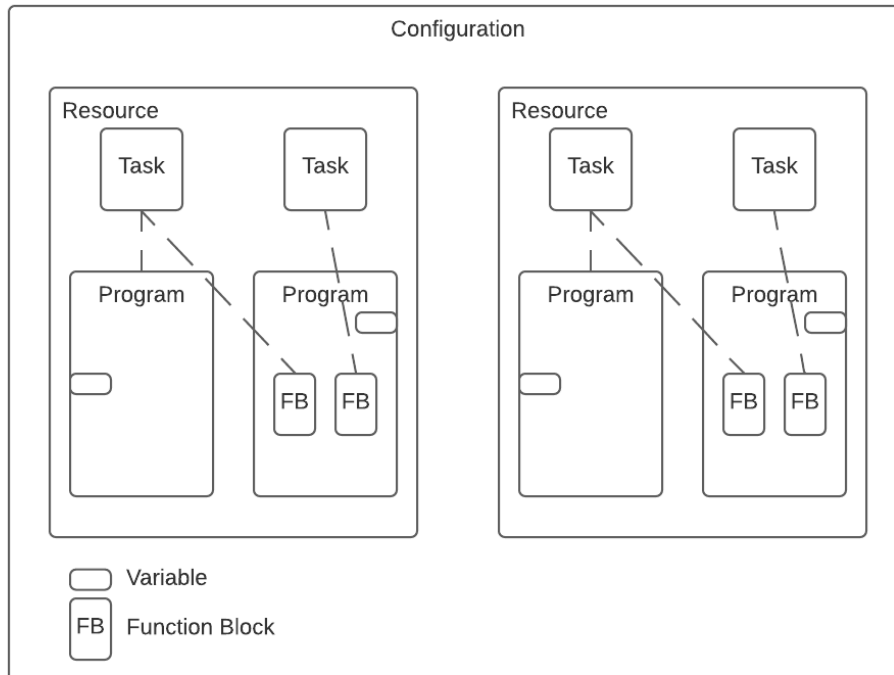


Figure 2.1: Structure of an IEC project. Based on [5]

2.1.1 Sequential Function Charts

SFCs are state machines used for the top-level sequential control of control processes. They consist of steps, transitions and actions. These elements provide a means of partitioning a PLC POU into a set of steps and transitions interconnected by directed links. With each step a set of actions, and with each transition a transition condition is associated [5].

Steps

Steps represent the phases of a control process. Each SFC has to have exactly one initial step, which is the entry point for the control sequence. Note that the initial step is drawn as a double-bordered rectangle.

Transitions

Transitions are the links between steps. They have a boolean fire condition which, when equalling true, transfers the status of the control sequence from the previous to the next connected step(s). Note that transitions can connect multiple steps with multiple steps in order to execute phases of the control sequence in parallel. Yet, the parallel sequences must be merged again in the process of execution. This particular case can be imagined as a token being passed around between the

steps. There is exactly one token in an SFC, but in the case of parallel execution it can be split in half. In order to be able to pass on the token after the parallel part of the SFC has finished, this token has to be merged again. Hence it is not possible to create an SFC in which a parallel part does not merge later in the sequence. We will cover this in more detail with an example in Section 2.1.2.

Actions

Actions are the executing part of the SFC. They are connected to steps, in which their content is to be executed. Inside an Action all of the five programming languages mentioned in Section 2.1 can be used.

2.1.2 SFC examples

In Figure 2.2 an example for a basic SFC is depicted. It is the control sequence of an elevator cabin, and the used graphical representation is taken from KEBA's current tool for developing IEC applications. Here one can also see how the different elements have to look in their respective graphical representation. Steps are drawn as rectangles, transitions are drawn as straight lines between the actions and a black box in the center of the line, where the name of the transition is displayed next to. Actions are depicted as connected boxes to their associated steps. The sequence starts in the initial step *Wait*. The step's actions are executed and when the transition *Trans1* fires, the step *CloseDoor* gets activated, and so on and so forth.

This basic example lacks complexity, yet shows the concept of SFCs. The graphical representation of SFCs obviously gets more viable with more complex control sequences. Figure 2.3 shows the SFC of the control of how the elevator cabin moves. In this case we see a parallel execution taking place. All the time while the moving of the cabin is controlled, the monitoring of the door is happening at the same time. The concept of merging the token explained before means that the transition *Trans3* can only fire if the second branch of execution is in the step *EndRide*.

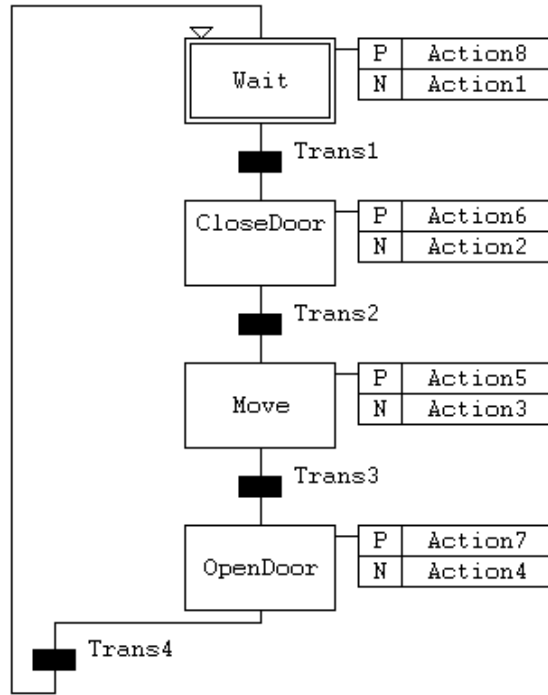


Figure 2.2: Example SFC: Top level control of an elevator cabin

2.2 Visual Studio Code

Visual Studio Code (VSCode) is an MIT licensed open-source source code editor [8] released by Microsoft in April 2016. It is based on the Electron framework, which uses the Chromium rendering engine with a Node.js runtime. It includes the concept of the Language Server Protocol (LSP), which will be covered in Section 2.3.

2.2.1 VSCode's Extension API

To make the basic VSCode framework more extensible, Microsoft added the possibility to develop so-called extensions. The extension API offers a wide range of possibilities to adapt the behaviour of VSCode. From supporting additional languages to simply customizing the UI. The most relevant type of extension for this thesis, however, are so-called webview extensions, which are used to build custom webpages inside VSCode and therefore utilize everything that web development technologies offer. This will then be the entry point for the used layouting kernel and more importantly the rendering library which were mainly developed for the use in a web browser.

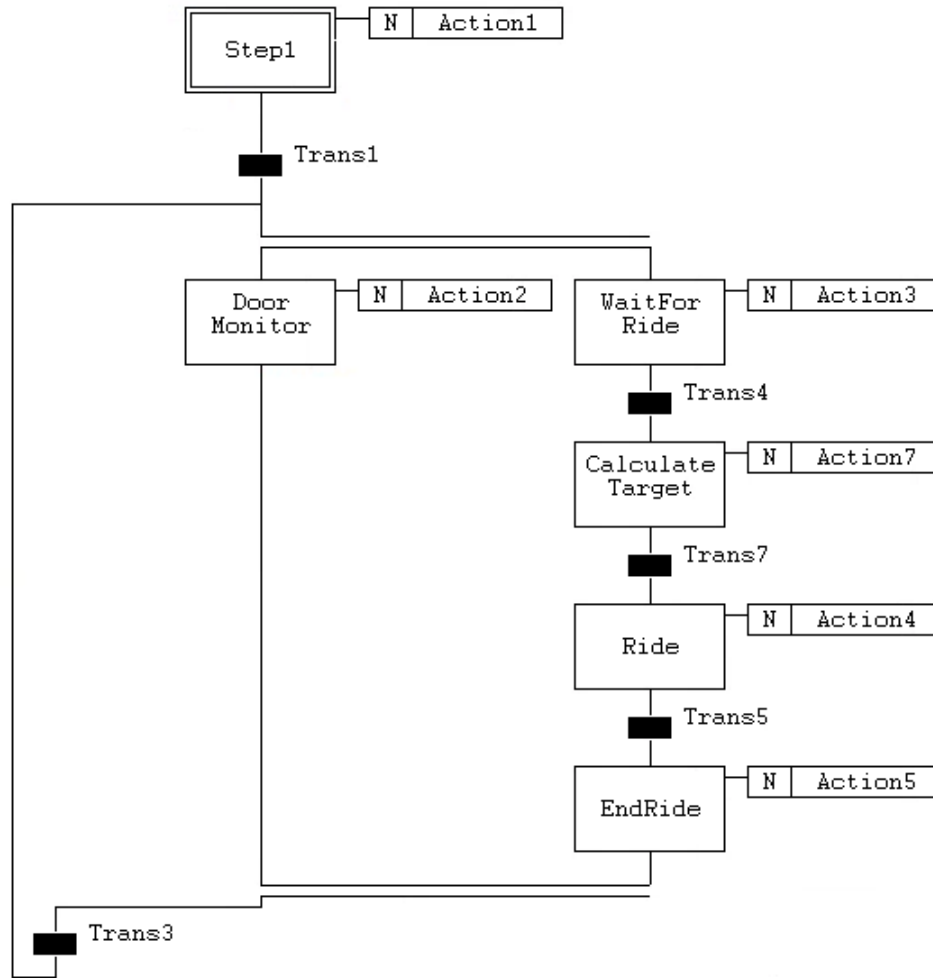


Figure 2.3: Example SFC: Top level control of an elevator ride

2.3 Language Server Protocol

The following is based on [6]. Implementing an editor capable of language features, such as diagnostics (e.g. displaying errors and warnings), a goto definition call or an autocomplete functionality, is arbitrarily complex. Traditionally this has to be done every time a new tool for a specific language is developed. The concept of language servers, clients and a corresponding communication protocol between them eliminates this problem. A language server is implemented to provide all the language specific smarts, which can then be queried by a client over a standardized language server protocol. It is worth mentioning that the protocol itself is JavaScript Object Notation (JSON) Remote Procedure Call (RPC) based, hence, it is a human-readable protocol which can be debugged easily.

2.3.1 Textual Language Server Protocol

For textual languages the protocol is already well defined. The communication between the client and the server starts upon opening the document, which lets the server know the Uniform Resource Identifier (URI) of the currently opened source file. Note that the client does not need to know which programming language was used to write the containing program. It simply calls the language server for the needed information every time a language specific action is taken. Figure 2.4 shows an example for the communication between language server and client. The client opens a document, edits it and executes "Goto definition". The server then reacts accordingly providing the needed information, diagnostics (i.e. compiling errors, etc.) or a position in the file, respectively.

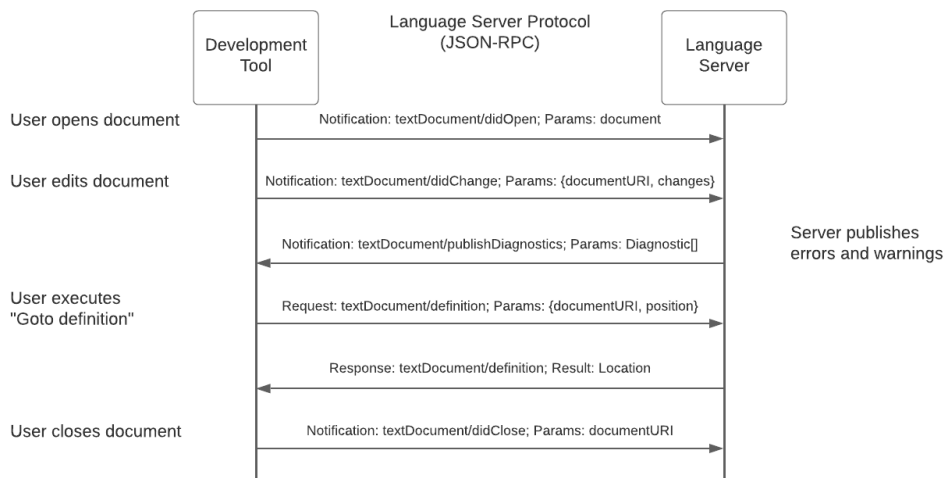


Figure 2.4: Client-Server communication via language server protocol. Based on [6]

2.3.2 Graphical Language Server Protocol and Intermediate Representation Format

To extend the LSP concept to graphical language representations some additional steps are required. In [13] the authors offer an approach of how to use the concepts of the LSP in the context of Graphical Modeling Languages. Since the main goal of this thesis is to automatically generate the graphical representation of a textually available (and compileable) SFC source code of the well-defined IEC 61131-3 (IEC) standard, not all of the presented concepts are usable for us. Yet, the proposed representation of the graphical information proves to be useful.

The idea is to create an Intermediate Representation Format (IRF), a JSON based format which represents the graphs structure. The definition of such a format is inspired by [10]. We will use this concept in Chapter 4, to get structural information about the SFC graph. Additionally this IRF is the basis for extending the viewer to an editor.

2.4 Graph Drawing

Since we have defined the look and feel of SFCs, we need to elaborate how an automatic generation of a layout of an SFC can be generated. Graph drawing algorithms can be the key to solving this problem. In general these algorithms are a vast field of research, the following sections will merely cover a very basic introduction to the ideas of graphs and drawing them as well as a brief review of the layered algorithm, which best covers the needs of drawing an SFC.

2.4.1 Theoretical Background

A number of data presentation problems involve the drawing of a graph on a two-dimensional surface. Usually the vertices are represented by symbols such as circles or boxes, and each edge (u, v) is represented by a simple open curve between the vertices u and v . A graph drawing algorithm reads a combinatorial description of a graph G as input, and produces a drawing of G according to a given graphic standard as output. The usefulness of such a drawing depends on its readability, that is, the capability of conveying the meaning of the presented data quickly and clearly. Readability issues are expressed by means of aesthetics, which can then be formulated as optimization goals for the drawing algorithms. A fundamental and classical aesthetic is for example the minimization of crossings of edges [1, pp. 20-27].

2.4.2 Layered Algorithm

Acyclic digraphs are widely used to display hierarchical structures. These graphs are laid out in a way that all edges point in the same direction, e.g. from top to bottom or left to right. A hierarchical drawing of an acyclic digraph is a polyline drawing where the vertices and bends are constrained to lie on a set of equally spaced horizontal lines, called layers. In Figure 2.5 such an acyclic digraph in upward direction is shown. In [14] an approach is presented to draw

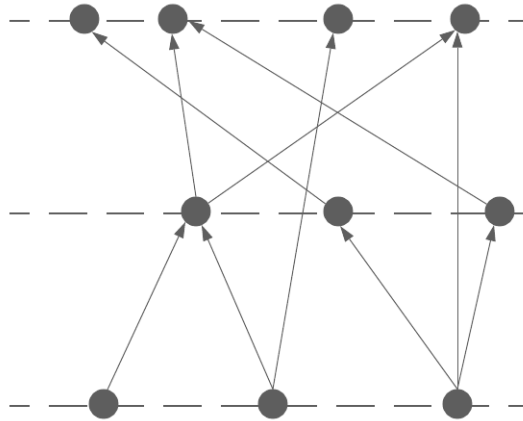


Figure 2.5: Drawing of a hierarchical acyclic digraph in upward direction. Based on [1, p. 22]

and optimize such a hierarchical digraph, which is commonly referred to as layered algorithm. As stated before only the basic ideas of the algorithm will be presented here, as the underlying mathematical concepts would take a much longer digression.

The algorithm is roughly partitioned into four Steps. Following is based on [14]:

- Step I: A "proper" hierarchy is formed from a given set of directed pairwise relations among elements of a system. If the digraph generated from the relations has cycles, it is condensed and we obtain a multilevel digraph or a hierarchy. Then, if the hierarchy has long span edges, it is converted into a proper hierarchy by adding dummy vertices and edges.
- Step II: The number of crossings of edges in the proper hierarchy is reduced by permuting orders of vertices in each level.
- Step III: Horizontal positions of vertices are determined. The order of the vertices determined in Step II is given as constraints to preserve the reduced number of crossings.
- Step IV: A (two-dimensional) picture of the hierarchy is automatically drawn where the dummy vertices and edges are deleted and the corresponding long span edges are regenerated.

Figure 2.6 shows graphically what happens in those four steps.

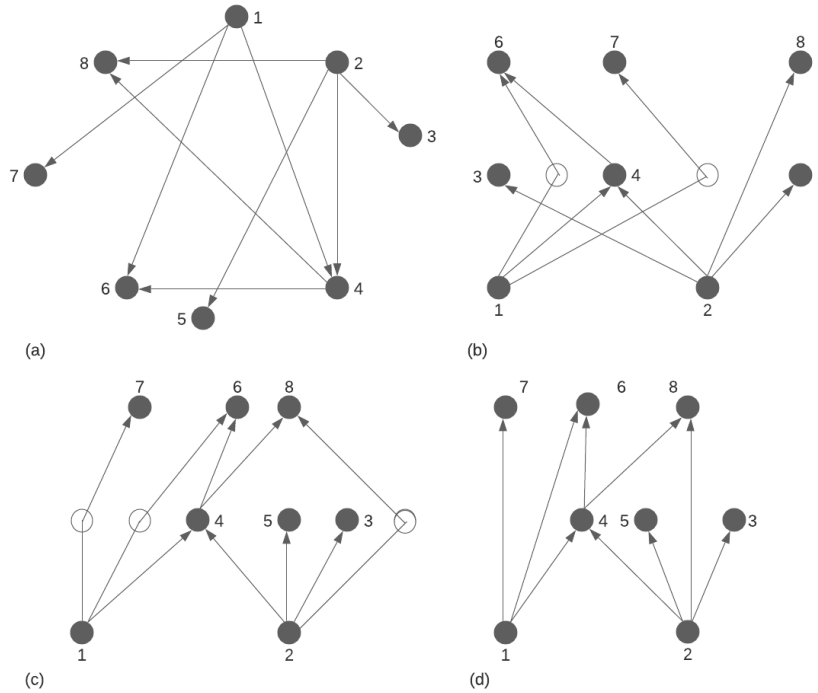


Figure 2.6: (a) Given digraph. (b) Assignment of vertices to layers (Step I). (c) Crossing reduction (Step II). (d) Placement of vertices and bends (Steps III and IV). Based on [1, p. 24]

2.4.3 Eclipse Layout Kernel

The Eclipse Layout Kernel (ELK) implements an infrastructure to connect diagram editors or viewers to automatic layout algorithms. It also implements a number of standard layout algorithms ready to be used out of the box [2].

Figure 2.7 shows the architecture of ELK. A diagram editor sends the graph information to the kernel which then applies a standard layout algorithm and sends position information of the vertices and edges back to the editor. Note that the drawing of this position information has to be done by the editor, hence a suitable drawing framework has to be chosen, which we will cover in Chapter 4.

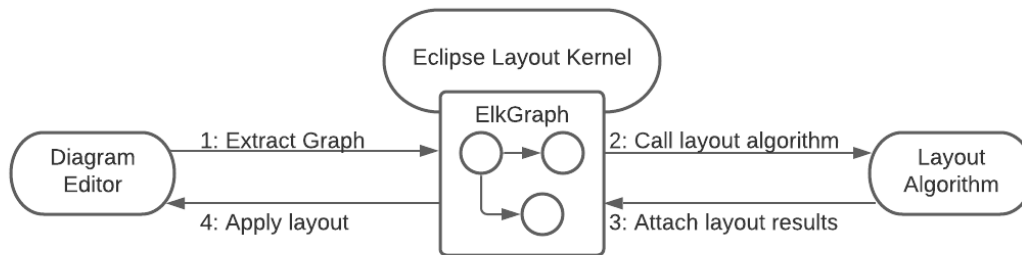


Figure 2.7: ELK architecture. Based on [2]

2.5 Summary

In this chapter we covered the basics of the IEC 61131-3 standard. Sequential Function Charts are the part of the standard which will be the main focus of this thesis. In order to draw the SFCs, a Visual Studio Code extension has to be implemented, and the basic Language Server Protocol features are to be extended in order to provide graphical information in the form of a so-called Intermediate Representation Format. For laying out the SFCs deterministically the layered algorithm of the Eclipse Layout Kernel is used.

The basics for the development of the SFC viewer are now set, in the following chapters we will focus on the requirements engineering by performing a Use-Case Analysis for the project. We will also take a look at different drawing engines in order to find the best suited one for our purpose.

Chapter 3

Concept and Analysis

In this chapter we will take a look at the methodical approach for finding the requirements presented in [9, pp. 39-114]. With the acquired knowledge we can then formulate the conceptual ideas and build the system architecture. For the visualization of the different diagrams the Unified Modeling Language (UML) will be used, which is a standardized graphical modelling language for designing systems.

3.1 Use Case Analysis

The use case analysis is divided into several parts which ultimately leads to a well-defined set of use cases and their respective requirements. First the system context and the actors of the system need to be defined. In the next step we need to find the relevant system use cases. These use cases will be set into relation to each other with the use of a use case model which will be refined by defining secondary use cases, which are basically tasks common to two or more system use cases. In a next step the use cases are analysed by the means of creating and refining their respective activity diagrams. This refinement will lead to basic test cases, which can be tested automatically. Finally the requirements for the system can be gathered in a systematic way.

3.1.1 System Context and Actors

The first step in this analysis is to define the system context and its actors. The system context sets the bounds for the to be developed system and defines the interfaces to third-party systems. In Figure 3.1, the system context diagram, the users and third-party systems are shown, UML calls them all actors.

We defined following actors in the process:

- IEC Language Server
The language server provides the structural information about the SFC
- Eclipse Layout Kernel (ELK)
ELK performs a layout algorithm on the given data.

- Drawing Framework
A suitable drawing framework draws the layouted graph.
- VSCode
VSCode is the underlying editor for the nested functionality.
- Application Engineer
An application engineer produces IEC code base applications, which will be extended by the external Developers to fit their needs.
- External IEC Developer
External IEC Developer extend the existing code base to fit the needs of their end-customers.
- End-Customer IEC User
The End-Customer IEC users use the IEC code of the external IEC Developers to run their actual industrial application (e.g. conveyor belt, molding machine, etc.).

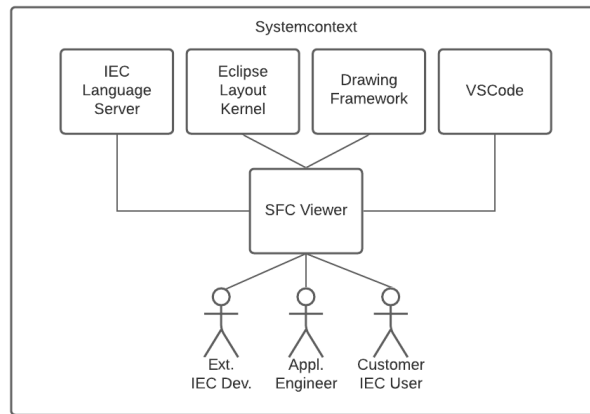


Figure 3.1: System context for the SFC Viewer.

3.1.2 System Use-Cases

After the system context is defined, we need to find the use cases of the system and how the defined actors are interacting with them. Figure 3.2 shows the found use cases in a use case model.

The external IEC developers and the application engineers mainly want to use the SFC Viewer as a help for developing the SFCs, therefore they mainly interact with the use cases concerning developing. The costumer IEC users on the other hand mainly focus on viewing and understanding the SFC and have a need to export the displayed SFC for documenting and sharing it, hence the use case *Export SFC* is needed.

The descriptions for the use cases will be documented in tabular form for readability.

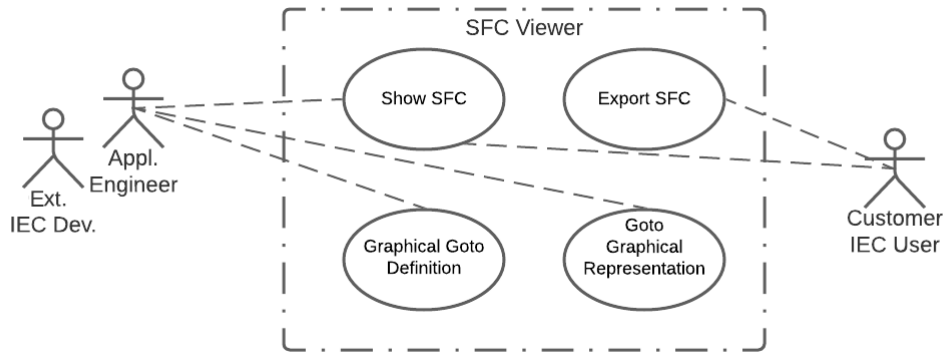


Figure 3.2: Use Case Model for the SFC Viewer

Show SFC

Name	Show SFC
Short description	Show graphical representation of a given textual SFC
Actors	IEC Developers, Language Server, VSCode, Drawing Framework
Trigger	User executes the Show diagram command for the displayed textual SFC
Preconditions	Textual SFC source code opened in VSCode
Incoming information	IRF from language server
Results	Graphical SFC is shown
Postconditions	none
Procedure	1. Get structural information from Language Server 2. Convert to ELK readable format 3. Layout with ELK 4. Open webview in VSCode 5. Draw graph with drawing framework

Export SFC

Name	Export SFC
Short description	Export a displayed graphical SFC
Actors	IEC Developers, VSCode, Drawing Framework
Trigger	User executes the export diagram command for the displayed textual SFC
Preconditions	Graphical SFC displayed
Incoming information	none
Results	Graphical SFC is exported
Postconditions	Exported File
Procedure	1. Open file browser 2. Save Scalable Vector Graphics (SVG) in desired location

Graphical Goto Definition

Name	Graphical Goto Definition
Short description	User wants to see the textual representation of a graphical element (Step/Action/Transition)
Actors	IEC Developers, Language Server, VSCode, Drawing Framework
Trigger	Click on an SFC element
Preconditions	Graphical SFC displayed
Incoming information	none
Results	Corresponding textual element is selected in the text editor
Postconditions	none
Procedure	1. Set background of the selected element in the graphical view to its defined selection color 2. Get structural information from Language Sever 3. Map structural information to actual code element 4. Select corresponding line in the text editor

Goto Graphical Representation

Name	Goto Graphical Representation
Short description	Highlight the graphical representation of a textual element (Step/Node/Transition)
Actors	IEC Developers, Language Server, VSCode, Drawing Framework
Trigger	Execute Goto Graphical Representation command
Preconditions	Graphical SFC displayed
Incoming information	none
Results	Corresponding element is selected
Postconditions	none
Procedure	1. Get structural information from Language Sever 2. Map structural information to actual code element 3. Highlight the corresponding graphical element

3.1.3 Redundant Steps

Some of the found use cases have redundant steps, which can be described as secondary use cases. These will then be connected with the original use cases with *include* relations. The secondary use cases created are *Get structural information from Language Server* and *Map structural information to actual code element*. *Export SFC* and *Graphical Goto Definition* need *Show SFC* as a base condition to work. This is depicted as arrows in Figure 3.3.

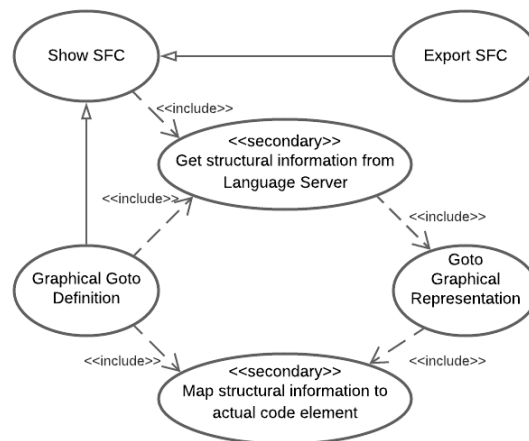


Figure 3.3: Refined Use Case Model for the SFC Viewer

3.1.4 Activity Diagrams

The next step is to define activity diagrams for the found use cases. This is done in an iterative approach.

- Describe standard sequence
- Refine each step of the standard sequence
- Derive test cases

We will execute this process exemplary for the *Show SFC* use case.

Standard Sequence

The activity diagram shown in Figure 3.4 visualizes the main steps defined in the use case. First, the structure of the SFC needs to be fetched from the language server. After that, the webview is generated and opened in VSCode, and the Conversion of the SFC structure is started in parallel. When the layout of the graph finishes and the webview is opened, the graph can be drawn, and the activity diagram finishes with the result *SFC shown*. Note, that in this first depiction of the use case activity diagram no erroneous behaviour is modelled. This will be refined in the next steps of the analysis.

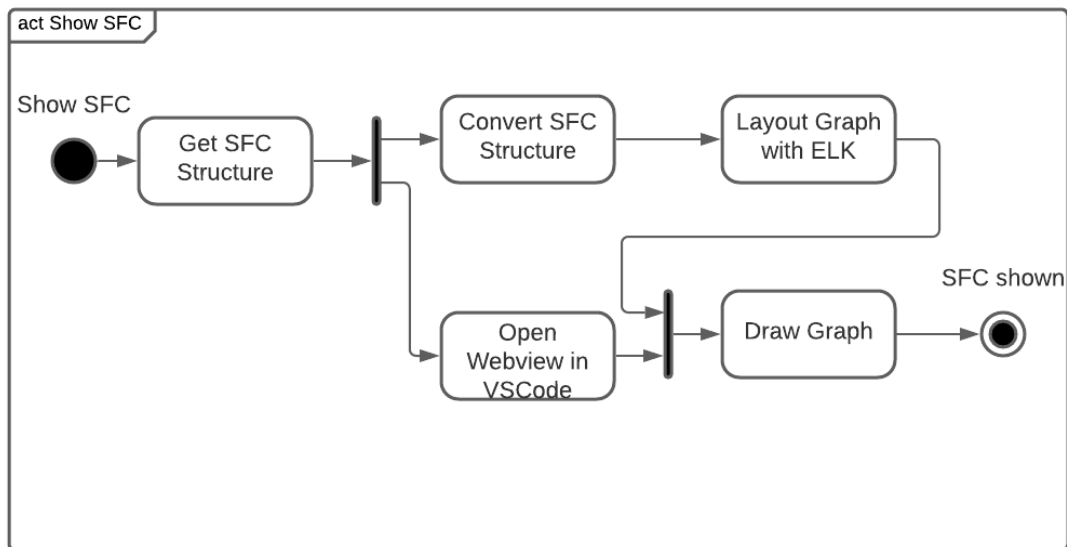


Figure 3.4: Standard Sequence of use case *Show SFC*

Refine Steps

The presented refinements of the individual steps can be seen in Figure 3.5. The main difference is now that each step has a fail condition as well. After combining it with Figure 3.4 the resulting activity diagram of the use case *Show SFC* can be seen in Figure 3.6.

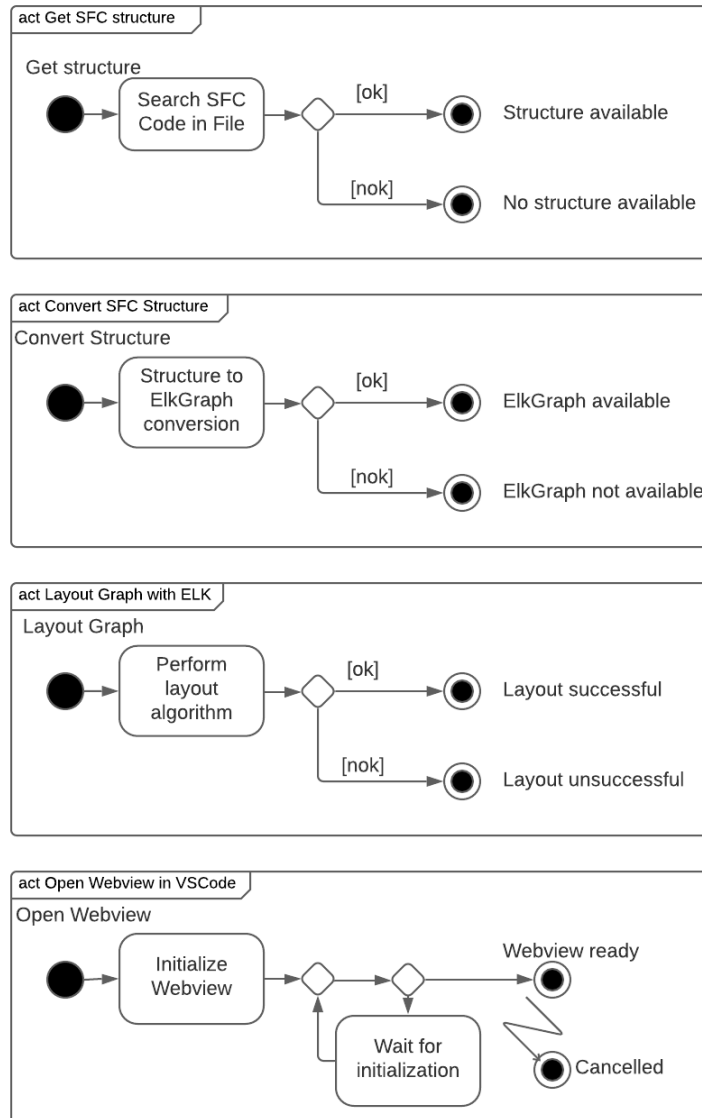


Figure 3.5: Refined steps

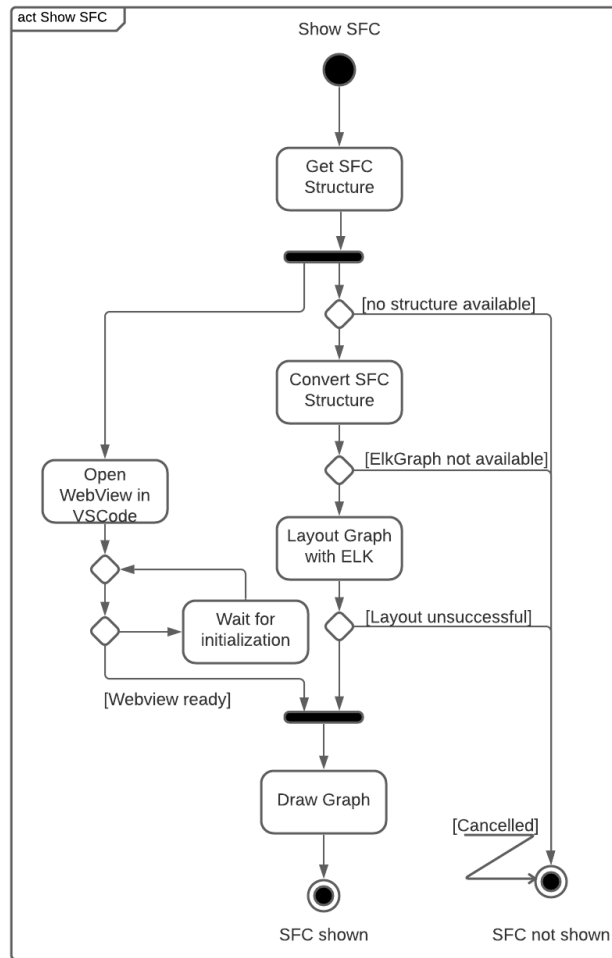


Figure 3.6: Completed sequence of use case *Show SFC*

Derive test cases

From this improved activity diagram we are able to derive basic failure cases, which are defined as follows:

- No SFC in the currently opened file
- Structural information has errors
- Derived ElkGraph can not be layouted
- Initialization of webview is cancelled

These failure cases can be tested automatically.

3.1.5 Requirements

With all the use cases and their respective failure cases well described, we are able to define the requirements of the system. For the sake of readability this will again be done in tabular form.

Requirement	SFC Look and Feel
Kind	Functional Requirement
Description	The SFC has to look as similar as possible to the standard
Stability	stable
Commitment	Must-Have
Priority	high
Details	The SFC has to comply to the IEC standard
Involved Elements	Show SFC
Requirement	Consistent Layout
Kind	Functional Requirement
Description	The layout needs to be consistent
Stability	stable
Commitment	Must-Have
Priority	high
Details	Every time the layout is performed, it must yield the same consistent result
Involved Elements	Show SFC
Requirement	Export SFC
Kind	Feature
Description	Export the SFC as picture
Stability	stable
Commitment	Should-Have
Priority	medium
Details	For documentation and/or sharing of SFCs
Involved Elements	Export SFC
Requirement	Goto Graphical Representation
Kind	Feature
Description	Highlight the element in the graphical SFC
Stability	stable
Commitment	Must-Have
Priority	medium
Details	The users need to keep track of their development, hence it is important to be able to show where the coded element results in in the graphical representation
Involved Elements	Export SFC

Requirement	Graphical Goto Definition
Kind	Feature
Description	Highlight the element in the textual SFC
Stability	stable
Commitment	Must-Have
Priority	medium
Details	The users need to be able to see the code of the resulting graphical element
Involved Elements	Export SFC

In reality this theoretical approach of gathering requirements is done in an issue tracking software at the company, hence this list is by default incomplete and constantly extended while further testing results and costumer feedback are coming in.

3.2 Concept and Architecture

Now that we analysed the system we can build a basic concept and an architecture of how the SFC Viewer should work.

3.2.1 Concept

Conceptually the SFC viewer can be divided into several layers according to a standard 3 layer approach as described in [9, p. 122]:

- Presentation layer
- Logic/Data layer
- Persistence layer

In the presentation layer we will use the VSCode webview to display the layouted graphs, with a fitting graph framework. The persistence layer, in our case the language server, is the part of our system which holds all the needed information for creating an SFC. The logic or data layer is responsible for mapping the information we get from the persistence layer, in such a way that the presentation layer is able to use it accordingly.

Figure 3.7 shows these graphically.

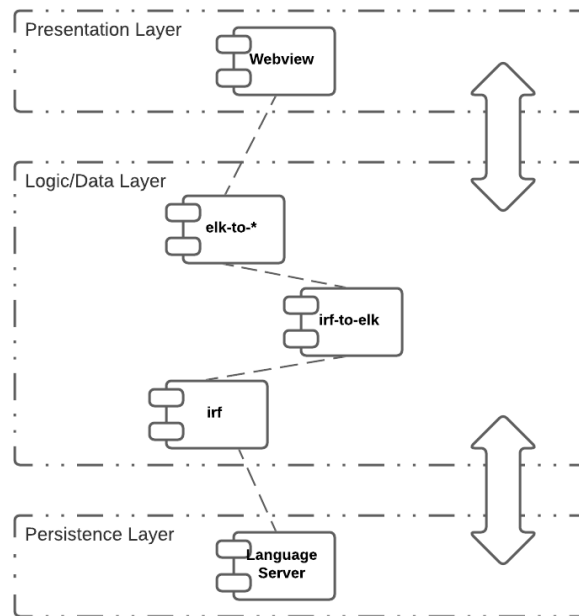


Figure 3.7: Layer model of the SFC viewer

3.2.2 Architecture

There are several levels of detail in which we can view the architecture of the SFC viewer. In Figure 3.8 the top-level architecture can be seen. The SFC Viewer extension is built on top of two already existing extensions. The base extension installs the Language Server on the system, whereas the IEC extension performs the basic language client actions. On top of that the SFC Viewer extension adds the functionality for creating graphical representations of SFCs. The extended LSP communication is in fact done by sending the IRF mentioned in Section 2.3.2.

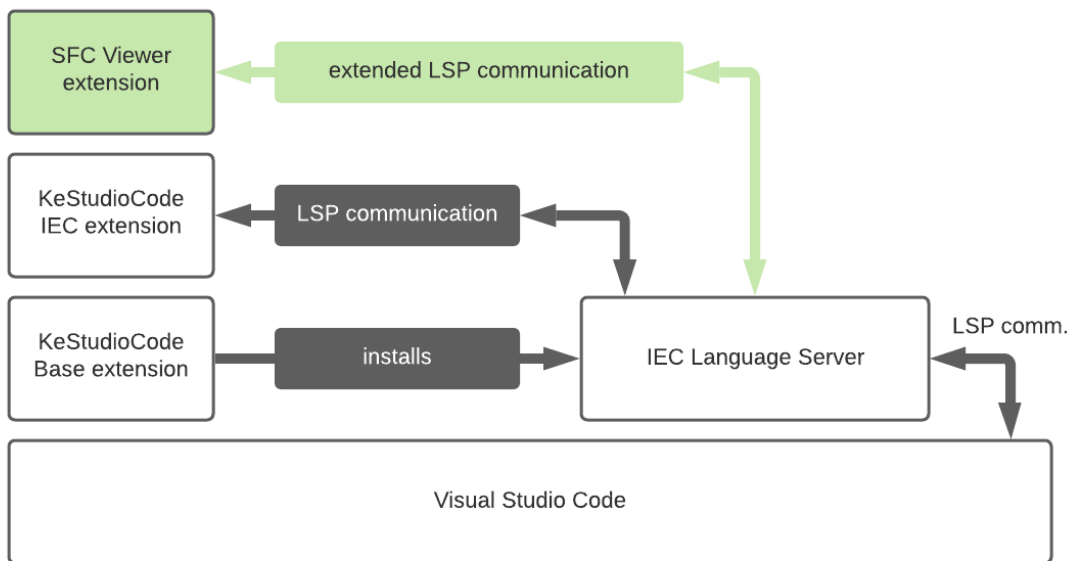


Figure 3.8: Top level architecture of the SFC viewer

Inside the SFC Viewer extension, in the next level of detail, the architectural considerations roughly follow the conceptual layers of Section 3.2.1.

As the drawing framework decision is not yet made, the architectural form of the presentation layer is yet to be found. For the logic/data layer the architecture can be seen in Figure 3.9. It links the conceptual components to the task they are assigned to.

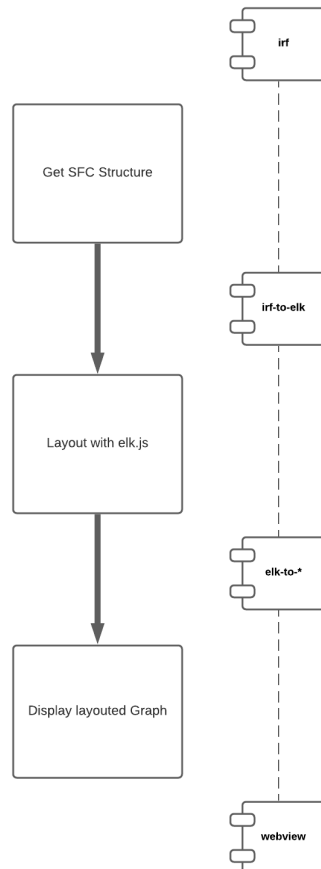


Figure 3.9: Architecture of the Logic/Data Layer

Chapter 4

Implementation

In this chapter the selection of the drawing framework will be done and the most important parts of the realization of the SFC Viewer will be outlined. The architectural properties of the chosen diagramming framework will be presented as well.

4.1 Comparison of drawing frameworks

While researching ways to draw graphs inside a VSCode webview, a number of JavaScript compiled libraries were found that potentially could have done the job. In this section we will take a look at the three most promising options, and decide which framework will be best suitable for performing the given task of displaying a layouted SFC.

4.1.1 mxGraph

mxGraph is the underlying drawing framework for the well-known graphing framework draw.io. It has a JavaScript frontend and is able to process given graphs and draw them accordingly. Although it is probably the most prominent framework on this list, the experiments with it prove to be unsuccessful, because of the lack of layout algorithm support. Therefore a consistent, automatically created layout of the SFC nodes would not have been possible.

4.1.2 Cytoscape.js

A rather different story is the Cytoscape.js framework. It is an open-source JavaScript-based graph library which has access to the most common graph layout algorithms. In the beginning the framework looked very promising, and so a first proof-of-concept design for drawing the graph was produced.

In the process of implementing this concept, however, it became clear, that the underlying and supported layout algorithms were in fact compiled into the library and therefore not configurable enough to be suited for the task. Despite trying all available layout options the result was not nearly close enough to the actual look and feel of an SFC, therefore the use of this framework was discontinued.

4.1.3 Sprotty

The third framework was Sprotty. It is a open source diagramming framework based on web-technologies, built upon TypeScript and JSX, which is an embeddable XML-like syntax, that is meant to be transformed into valid JavaScript [7]. It uses Scalable Vector Graphics (SVG) for rendering the to be displayed components. These SVGs are supported by all modern browsers, therefore also for the electron browser which is the basis for VSCode’s webview.

Another convenience of the Sprotty framework is that its developers already included an example VSCode extension in their repository, which already poses a great starting point for the task at hand.

4.1.4 Selection

As a starting point for the decision making the following table should perform as an overview of the capabilities of each of the mentioned frameworks. With this information a thorough decision can be made upon which the implementation will be built upon.

	mxGraph	Cytoscape.js	Sprotty
web-tech-based	✓	✓	✓
Layout Algorithms	x	✓	✓
Communication	~	~	✓
Integration in VSCode	x	x	✓

As all of the frameworks are JavaScript or TypeScript based, they are suitable for the usage, in the webview context of VSCode. The second point which is important for the drawing framework is the fact if layout algorithms are included or supported, this is only the case in Cytoscape.js and Sprotty. As a third criterion for the suitability of the framework, the communication possibilities of the framework with its surroundings is evaluated. Only Sprotty has a pre-defined way of communicating with other parts of the software. For mxGraph as well as Cytoscape.js a own implementation of communication has to be built, which will have to be based on a REST-API or similar web technology standards. Lastly, the integration in VSCode is a big factor of deciding which framework to use. Only Sprotty has this feature already packed into the framework-base, the other frameworks need a lot of overhead to use in a VSCode context.

The Sprotty framework with all its features and in-built support for VSCode and ELK was the best choice. It also specialises on directed graphs such as Finite State Machines even in its examples, which as well put the decision making process in its direction.

4.2 Intermediate Representation Format

The first step of the realization was to define the IRF, which is key to developing the project. Since the language server itself only has the information about the SFC from the parser information of the IEC compiler, we need a way of serializing the SFCs structure. This way we can extract the necessary structural information of the SFCs and send them in a convenient way over the existing mechanisms of the LSP. The ideas for this format are inspired by [13] and adapted to the needs of this thesis.

The description of the IRF is done best from outer level to inner level, in Listing 4.1 the top-level structure is shown.

Listing 4.1: Top-level structure of IRF

```
1 {  
2   "version": "1.0",  
3   "language": SFC,  
4   "sfcName": "tCabinControl",  
5   "trans": [ ... ],  
6   "acts": [ ... ],  
7   "steps": [ ... ]  
8 }
```

On the top-level, the IRF contains the fields *trans*, *acts* and *steps*. These are arrays of their respective types contain all of the elements which need to be layouted and drawn.

All of the SFC elements in the IRF have a set of common entries:

- id: a unique identifier
- name: the elements name
- range: defines where the element is located in the source code file

Transitions have the following additional entries:

- from: the unique identifier of the element from which the transition originates
- to: the unique identifier of the element to which the transition leads
- isFirstAlternative: whether or not the transition is the first alternative transition
- isParallelSep: whether or not the transition gets separated into two or more parallel paths
- isParallelReunit: whether or not the transition reunites two or more parallel paths
- isGoOn: whether or not the transition is a go on transition
- isExit: whether or not the transition is an exit transition
- isEntryCode: whether or not the transition connects to a step which is in the exit code of the SFC

Steps have the following additional entries:

- `isInitStep`: whether or not the step is the initial step of the SFC
- `isInExit`: whether or not the step is in the exit code of the SFC
- `isEntryFromExit`: whether or not the step gets entered from the exit code of the SFC
- `actions`: a list of actions, which are associated with the step. In this action association array the action's unique ID is listed as well as the action's qualifier.

Actions have only one additional entry, which states if the action contains a sub-SFC or not.

Listings 4.2 to 4.4 show the JSON representations of the respective elements.

Listing 4.2: Transition structure

```
1 {
2   "id": 123456789,      // unique identifier
3   "name": "Trans1",
4   "range": { ... },    // reference to code
5   "from": 1,
6   "to": 2,
7   "isFirstAlternative": true/false,
8   "isParallelSep": true/false,
9   "isParallelReunit": true/false,
10  "isGoOn": true/false,
11  "isExit": true/false,
12  "isEntryCode": true/false
13 }
```

Listing 4.3: Step structure

```
1 {
2   "id": 123456789,      // unique identifier
3   "name": "Step1",
4   "range": { ... },    // reference to code
5   "isInitStep": true/false,
6   "isInExit": true/false,
7   "isEntryFromExit": true/false,
8   "actions":
9   [
10    {
11      "id": 123456789,
12      "qualifier": "P"
13    }
14  ]
15 }
```

Listing 4.4: Action structure

```

1 {
2   "id": 123456789,          // unique identifier
3   "name": "Action1",
4   "range": { ... },        // reference to code
5   "hasSubSfc": true/false
6 }

```

With this structure, it is possible to express all the information about the SFC in compact textual form, which can then be translated to be used as the graph information needed by ELK.

4.3 irf-to-elk Bridge

In order to feed ELK the information gathered via the IRF, it is necessary to convert it to ELKs own JSON format. It has five basic elements: nodes, ports, labels, edges and edge sections. A detailed description can be found in [3].

The most important thing for our conversion is the configuration of the layout options, which can be added as well to each of the above mentioned elements and to the top-level graph itself. Here we can define which pre-packed layout algorithm we want to use for layouting the graph and configure the selected algorithm. In our case we will take the layered algorithm, explained in Section 2.4.2, with the overall layout direction *DOWN*. Since the SFCs have steps with their associated actions in a left-to right direction, the overall direction of the edges between steps is different to the edges between steps and their associated edges. This is in fact a problem which is not trivial to solve for the layout algorithm. Therefore we need to build a composite node for each step in which a different layout algorithm direction can be run. These composite nodes are now the children of the top-level graph which are layouted in the *DOWN* direction. The original steps and their associated actions are put into the composite node as child nodes and layouted with the layered algorithm with the *RIGHT* direction. Figure 4.1 shows this approach graphically.

The next important thing is to consider ELKs ports as starting and end points for edges. It is the only way to fix the positions where edges can start or end. If these endpoints are not set, the layout algorithm will choose the position where to start the routing of the edges with respect to crossing minimization and other optimization criteria.

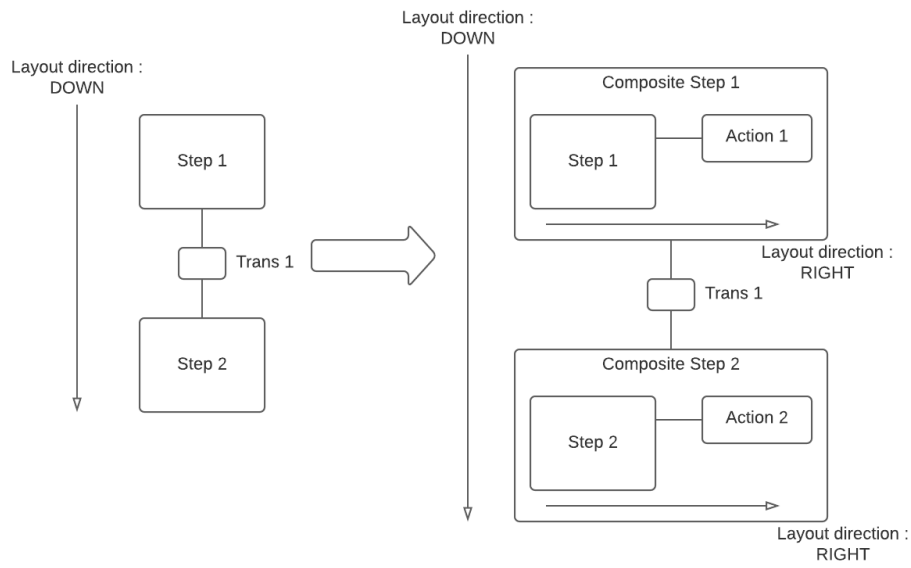


Figure 4.1: Use of composite node for changing layout direction

Since the transitions in an SFC have boxes in the middle of the respective edge it is necessary to split the resulting edges in two, separated by an additional node, which is shown in Figure 4.2. The ELK then layouts the given graph, and returns an `ElkGraph` object, which contains the layouted nodes and edges with their respective calculated positions.

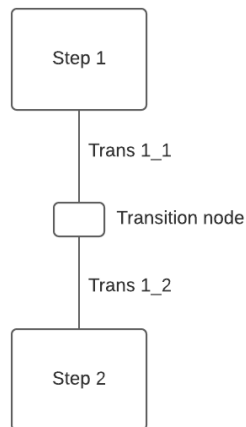


Figure 4.2: Split of transitions

4.4 Sprotty Architecture

4.4.1 Introduction

The Sprotty framework can be used as a client-only application that holds a model that can be rendered with some interactions with the user. But the more complete way also includes a server that maps an underlying model to diagram elements and handles more resource and memory intensive workloads that should not be executed on the client. [12]

Yet, in our case the client-only approach is the desired one, since the model does origin from the IEC language server and is converted to a Sprotty displayable format in the VSCode extension. The architecture on the client-side does not follow a traditional Model View Controller (MVC) approach, but rather a modern reactive framework, where information flows in a circle between three components. Messages to and from the server or within Sprotty are called actions that contain some kind of operation, such as requesting a diagram model or setting the bounds of some model elements. Internally they are then translated into commands to alter the diagram model state. The Sprotty Model (SModel) can be translated to different views registered in the view registry. These different views are comparable to an HTML Document Object Model (DOM) that is able to create a web page view like browsers would show it, or a tree view of all tags, their attributes, and children [12]. The client architecture can be seen in Figure 4.3.

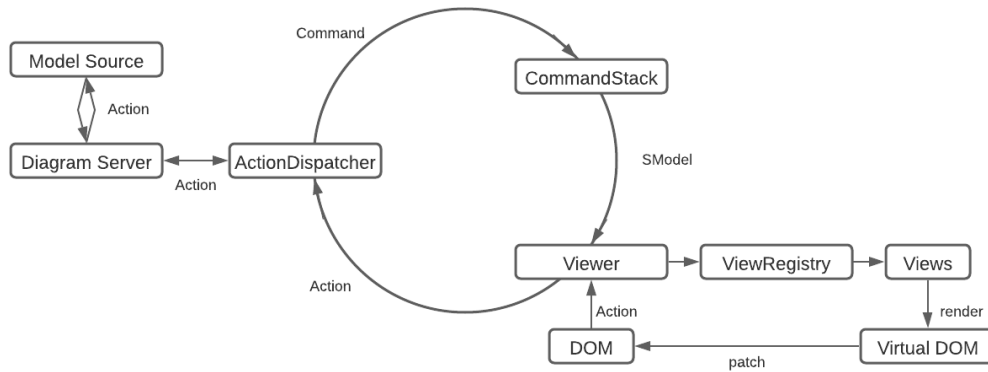


Figure 4.3: Sprotty client architecture. Based on [12]

The base architecture of Sprotty does however not presume that the given visualization is a graph with nodes and edges, therefore it includes a library which adds the abstraction layer of a graph to it. A Sprotty Graph (SGraph) contains Sprotty Nodes (SNodes) connected by Sprotty Edges (SEdges). A more detailed description of the inner workings of Sprotty can be found in [4].

4.4.2 Mapping Intermediate Representation Format-Eclipse Layout Kernel-Sprotty

For a deeper understanding of how the mapping between SFC source, IRF, ELK and Sprotty is done we will provide a minimal example.

Considering only a single initial step as SFC, we can illustrate how the SFC looks in its different forms along the way from source code to being displayed by Sprotty.

First the IRF has to be generated from the language server, which can be seen in Listing 4.5.

Listing 4.5: IRF of Single-Step SFC

```
1 {
2   "version": "1.0",
3   "language": "SFC",
4   "sfcName": "test",
5   "trans": [],
6   "acts": [],
7   "steps": [
8     {
9       "id": 8452750078242944000,
10      "name": "Step1",
11      "range": {
12        "start": {
13          "line": 5,
14          "character": 1
15        },
16        "end": {
17          "line": 6,
18          "character": 9
19        }
20      },
21      "isInitStep": true,
22      "isInExit": false,
23      "isEntryFromExit": false,
24      "actions": []
25    }
26  ]
27 }
```

Next it will be converted to a ELK-JSON object, see Listing 4.6. It has to be stated, that the width and height of the respective elements are hard-coded. A more flexible approach for the node sizes to be calculated was omitted, because of inconsistent results in terms of the overall look and feel of the resulting SFCs.

Listing 4.6: ELK-JSON object of Single-Step SFC

```

1 {
2   "id": "root",
3   "layoutOptions": {
4     "algorithm": "layered",
5     "elk.direction": "DOWN"
6   },
7   "children": [
8     {
9       "id": "8452750078242944000_comp",
10      "layoutOptions": {
11        "algorithm": "layered",
12        "elk.direction": "RIGHT"
13      },
14      "renderOption": "parent",
15      "children": [
16        {
17          "id": "8452750078242944000",
18          "width": 128,
19          "height": 64,
20          "labels": [
21            {
22              "id": "Step1",
23              "text": "Step1"
24            }
25          ],
26          "renderOption": "init",
27          "layoutOptions": { ... },
28          "ports": [ ... ]
29        }
30      ],
31      "edges": [ ... ],
32      "ports": [ ... ]
33    }
34  ],
35  "edges": [],
36  "labels": []
37 }

```

This object is then sent to ELK to be layouted, which returns an `ElkGraph` TypeScript object, which is best described with an UML class diagram, see Figure 4.4. The class `ElkGraph` implements the interface `ElkNode`, which itself is extended from the interfaces `ElkShape` and `ElkGraphElement`. The `ElkGraph` object capsules all the information of the ELK-JSON format in a TypeScript context, with which the algorithm is able to calculate the position and size of all the containing elements.

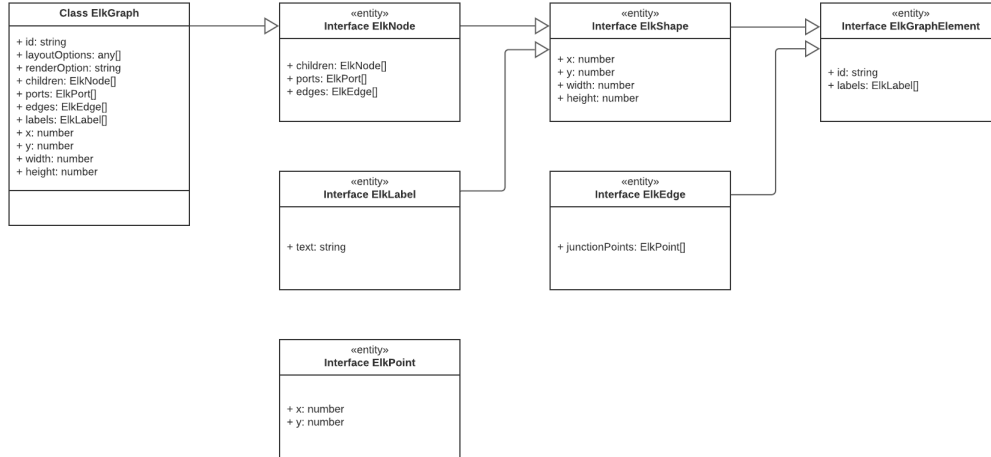


Figure 4.4: Class diagram of an ELK object

The conversion to the displayable `SGraph` is also done in TypeScript. It is mainly a one-to-one conversion of the `ElkGraph` object to the `Sprotty` data structure. The `ElkGraphJsonToSprotty` class contains a public method `transform` which returns the `SGraphSchema` (a serializable version of the `SGraph` class). It triggers the methods which perform the conversion, the method stubs can be seen in Listing 4.7.

Listing 4.7: `ElkGraph` to `SGraph` conversion methods

```

1 public transform(elkGraph: ElkNode): SGraphSchema;
2 private transformElkNode(elkNode: ElkGraph): SFCNodeSchema;
3 private transformElkPort(elkPort: ElkPort): SPortSchema;
4 private transformElkLabel(elkLabel: ElkLabel): SLabelSchema;
5 private transformElkEdge(elkEdge: ElkEdge): SFCEdgeSchema;

```

After this conversion the resulting `SGraph` object is ready to be sent to the `Sprotty` client to be displayed.

4.4.3 Extension integration

The next step is to integrate the Sprotty client in our VSCode extension. As already stated above, Sprotty has an example project, which is the base for integrating the client into an extension. It uses a language server connection to get the graphical information of a generic Domain-Specific Language (DSL). The also included language server of the example is designed to already deliver a drawable SModel, hence several adaptations have to be made in order to fit the SFC Viewers needs. Our extension has to fetch the SFC structure in IRF form from the IEC language server. Afterwards it performs the SFC conversion to a layouted graph in ELK format which has to be mapped to the above mentioned SModel/SGraph structure in order to be able to be displayed by Sprotty.

The Sprotty client itself lives inside a VSCode webview which communicates with the extension via VSCode's internal REST-API. The POST messages which are sent, contain Sprotty actions which can be interpreted by the client. This way we can for example send a *SetModelCommand* as Sprotty action, which renews the underlying model for the presented SFC. This architecture is shown in Figure 4.5.

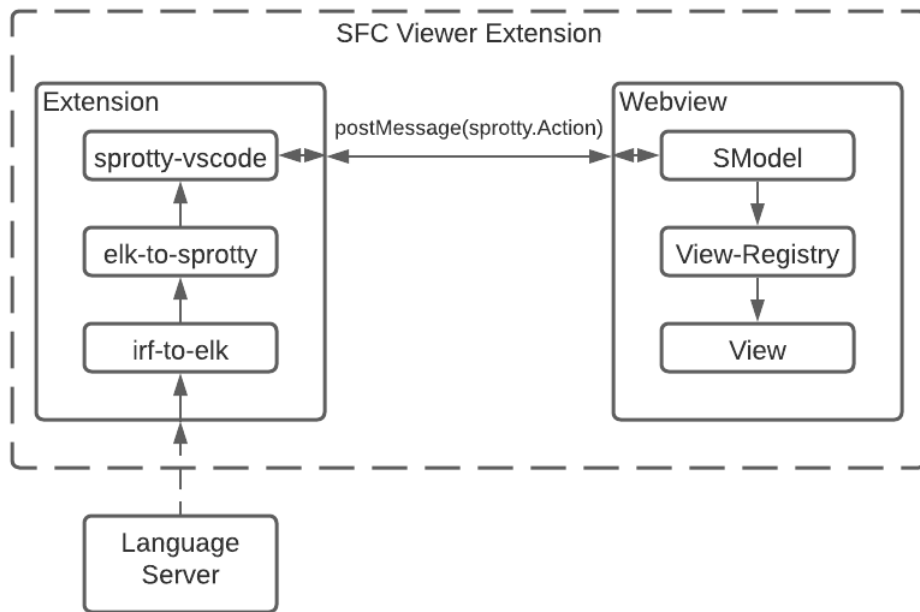


Figure 4.5: Extension architecture with sprotty

4.5 Realization

All of the above mentioned concepts have to be implemented inside a VSCode extension. The command *Show and fit in Diagram* triggers the language server and generates the ElkGraph object via the irf-to-elk bridge. This object is transformed to an SGraph, which is then sent to the webview via a Sprotty action. Inside the webview this SGraph is converted to an SModel which can then be mapped to a HTML DOM and finally be displayed as a webpage rendering of SVGs in the webview.

4.6 Summary

The result of applying all of these concepts was a first working prototype. Its functionality is to display a given SFC in a separate window besides the textual editor. Figure 4.6 shows a first glimpse at how the SFC Viewer looks like.

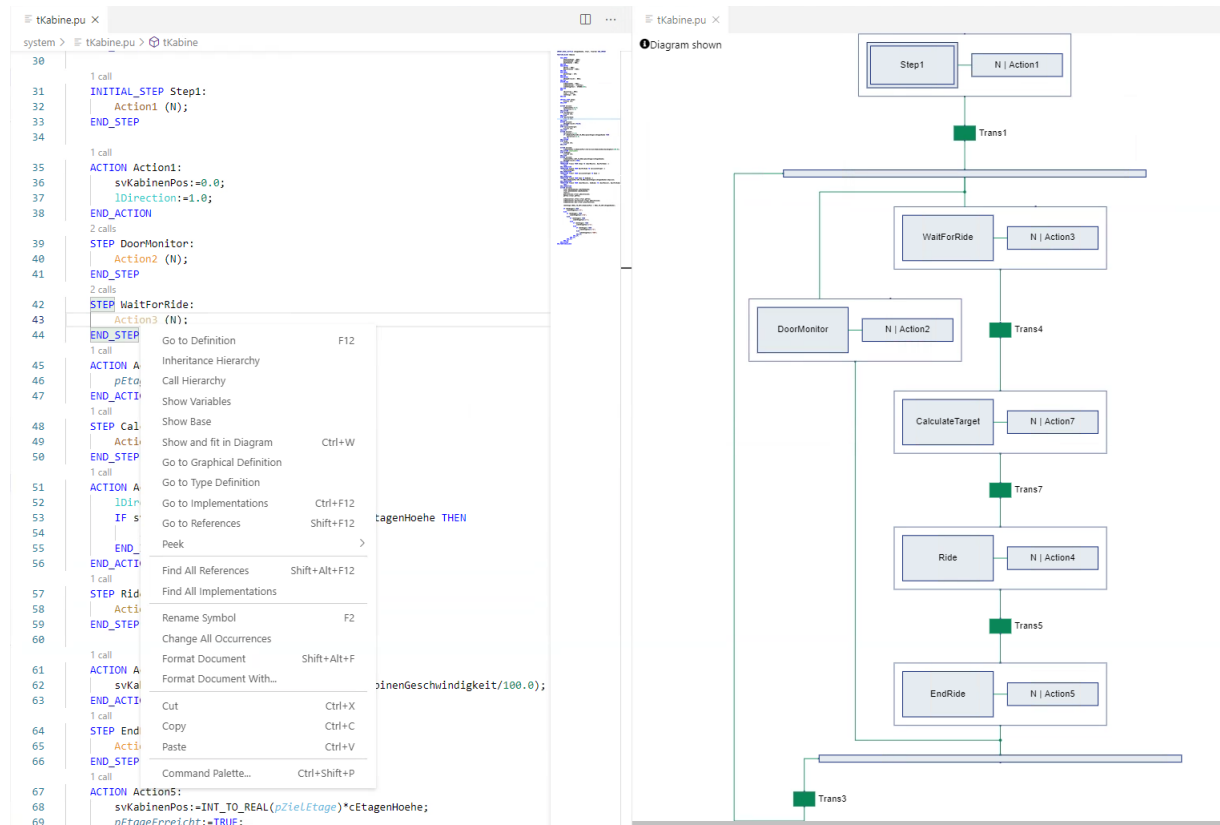


Figure 4.6: Prototype of the SFC Viewer

Chapter 5

Evaluation

In this Chapter we will take a closer look at the SFC Viewer prototype. First, the example SFCs of Section 2.1.2 will be examined. The look and feel must closely match the original examples. The next test will be the generation of a large SFC which will be evaluated for its viewability and usability as a help for the user. In the final phase of the evaluation we will incorporate feedback from different actors, i.e. real life users of the SFC Viewer, in order to adapt and extend the use cases, as well as extend the requirements for the SFC Viewer.

5.1 Elevator Cabin Control

First, the elevator cabin control example, seen in Figure 2.2, is examined. We chose this particular example because of its simplicity. It is an SFC with four steps, each of them containing two actions. The sequence is straight down with no parallel or alternative sequences and only one feedback edge. The first acceptable results of the early stages of the development process were achieved with this simple example. In Figure 5.1 the output of the SFC Viewer can be seen. The general structure is presented correctly, yet the composite nodes, which were necessary to be able to change the graph direction for the associated actions, are not compliant to the original look and feel.

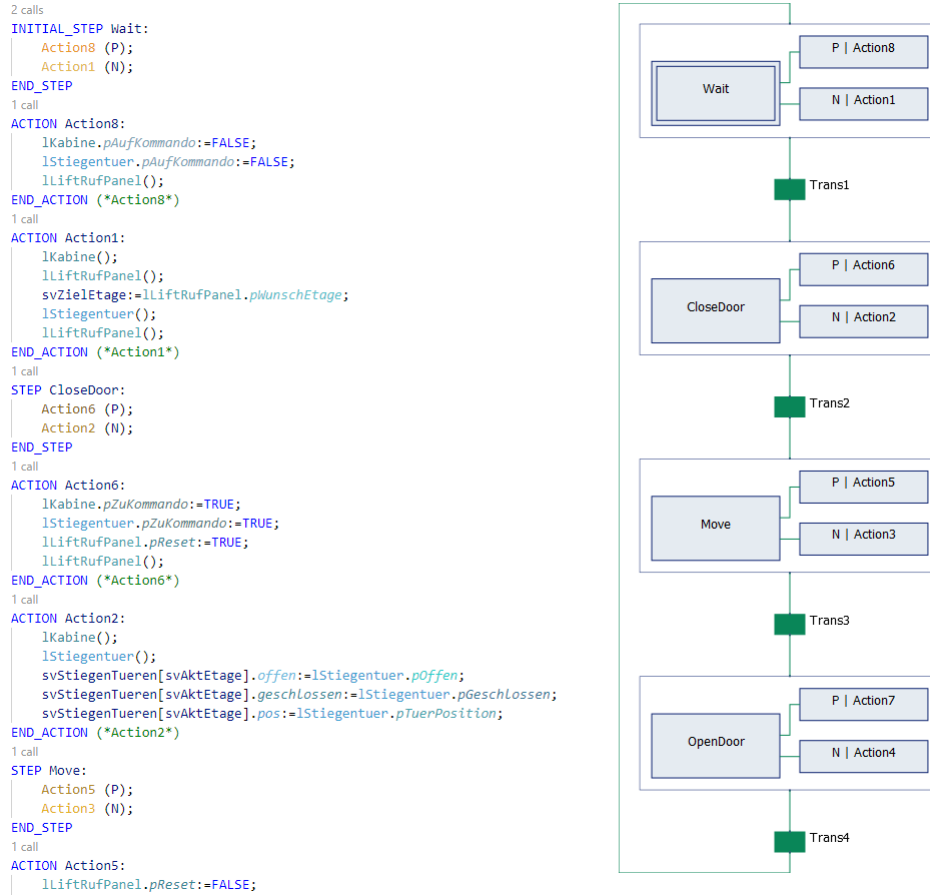


Figure 5.1: Example SFC: Top level control of an elevator cabin

5.2 Elevator Ride Control

As a second example, we took an SFC with parallel sequences, first shown in Figure 2.3. This example was mainly selected, because of the extensive changes to how the transitions had to be converted to the ELK-JSON format. In order to get closer to the original look and feel of the SFC, additional nodes had to be implemented at the beginning and the end of the parallel sequence. These nodes have to be added, if the associated transition has either the *isParallelSep* or the *isParallelReunit* flag set. It was indeed a long process until the placement and connection between the corresponding elements really worked.

The output for the second example can be seen in Figure 5.2. Again, the composite nodes and the layout of the actions inside seem to be the main differences to the original example.

```

1 call
INITIAL_STEP Step1:
| Action1 (N);
END_STEP

1 call
ACTION Action1:
| svKabinenPos:=0.0;
| lDirection:=1.0;
END_ACTION

2 calls
STEP DoorMonitor:
| Action2 (N);
END_STEP

2 calls
STEP WaitForRide:
| Action3 (N);
END_STEP

1 call
ACTION Action3:
| pEtageErreicht:=FALSE;
END_ACTION

1 call
STEP CalculateTarget:
| Action7 (N);
END_STEP

1 call
ACTION Action7:
| lDirection:=-1.0;
| IF svKabinenPos<INT_TO_REAL(pZielEtage)*cEtagenHoehe THEN
| | lDirection:=1.0;
| END_IF;
END_ACTION

1 call
STEP Ride:
| Action4 (N);
END_STEP

1 call
ACTION Action4:
| svKabinenPos:=svKabinenPos+(lDirection*cKabinenGeschwindigkeit/100.0);
END_ACTION (*Action4*)

1 call
STEP EndRide:
| Action5 (N);
END_STEP

1 call
ACTION Action5:
| svKabinenPos:=INT_TO_REAL(pZielEtage)*cEtagenHoehe;
| pEtageErreicht:=TRUE;
END_ACTION

1 call
TRANSITION Trans1 FROM Step1 TO (DoorMonitor, WaitForRide) :=
| TRUE;
END_TRANSITION

1 call
TRANSITION Trans4 FROM WaitForRide TO CalculateTarget :=
| pFahreInEtage;
END_TRANSITION

```

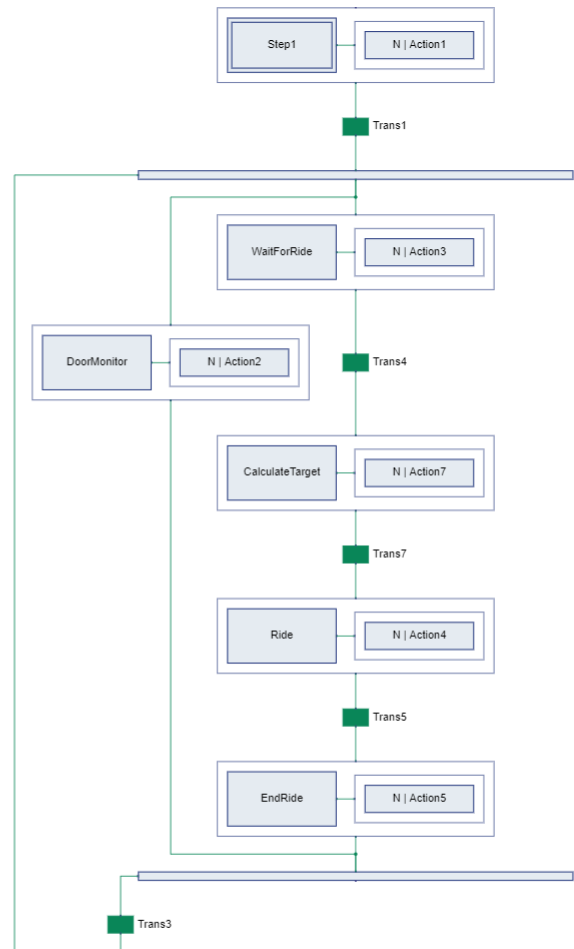


Figure 5.2: Example SFC: Top level control of an elevator ride

5.3 Layout of Large SFCs

Figure 5.3 shows a large SFC with many transitions pointing back to the initial step *Init*. This is a very common practice in IEC developing, many error cases or process finishes are modelled as transitions back to the start.

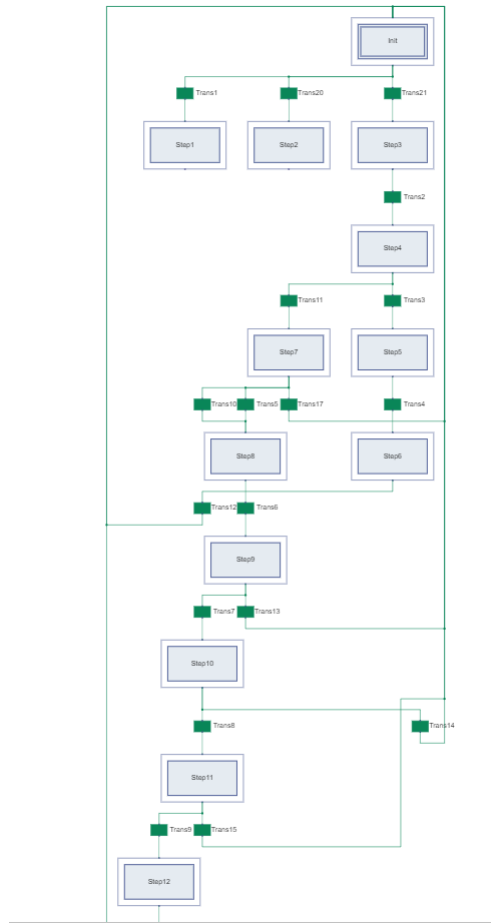


Figure 5.3: Example: Large SFC with many transitions back to *Init*

As this is a nuisance to layout and a non-relevant information, the original SFC editor introduced so-called *Goto Transitions*. These are marked as downward facing triangles and only state which Step they are going to. An example for *Goto Transitions* can be seen in Figure 5.4. In the case of the SFC Viewer this is currently not possible, since these types of transitions do not exist in the IEC standard and are therefore not existent in any kind of textual representation of the SFC.

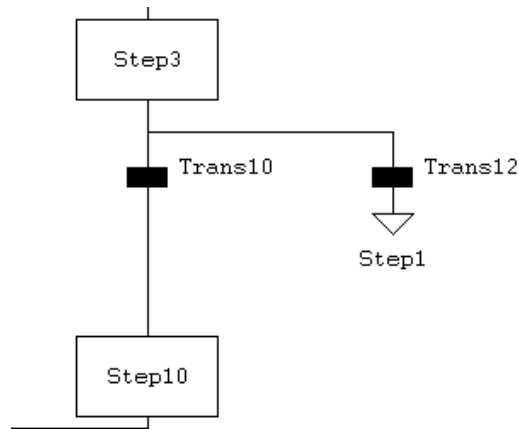


Figure 5.4: Goto Transition in old SFC editor

5.4 User Feedback

Since we defined several human actors in the use case analysis in Chapter 3 it is important to get feedback from them. In the next Sections the feedback of early tests will be processed. In-house application engineers as well as external IEC developers tested the SFC Viewer.

5.4.1 General Remarks

All in all the actors' feedback was positive, the users were excited that a new technology was the base for the heavily used tool. Yet, the overall usability and look and feel were the most-addressed topics in the feedback meetings. The biggest "problem" they saw, however, is the not existing editing functionality. Since this is not in the scope of this thesis we will not cover these remarks.

Generally said, the switch to a new tool is never easy, especially if the old tool was around for about twenty years. Therefore the feedback had to be filtered in order to find real improvements for the actual scope of the tool. In addition to that, the technical boundaries of the layout algorithm as well as of all the other used technologies have to be taken into account while incorporating the feedback.

5.4.2 Use Case Adaptations and Additional Requirements

The usability topics that came up in the meetings, especially with the external IEC developers, led to the discovery of additional use cases, which were not defined upfront. The users need the possibility to search for an element of the SFC inside the view, and switch between different detail levels in the viewer. This leads to the addition of the new use cases to the use case model:

- *Find Element in SFC*
- *Change Detail Level*

Figure 5.5 shows the updated and extended use case model. As a result of that, naturally, we get new requirements for the SFC Viewer, which will be defined in tabular form, as seen in Section 3.1.5

Requirement	Find Element in SFC
Kind	Feature
Description	Find the element in the graphical SFC
Stability	stable
Commitment	Must-Have
Priority	medium
Details	The users sometimes need to find any given graphical element as fast as possible. Thus, a find in SFC feature is necessary
Involved Elements	Show SFC

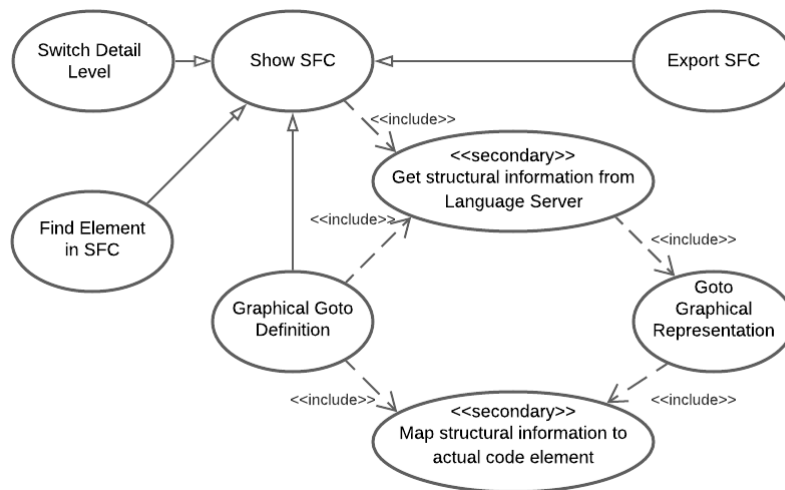


Figure 5.5: Extended Use Case Model for the SFC Viewer

Requirement	Switch Detail Level
Kind	Feature
Description	Hide or show certain elements in the SFC
Stability	stable
Commitment	Nice-to-Have
Priority	medium
Details	Different detail levels are necessary, sometimes only the overall structure of the SFC is of interest, sometimes actions and even inline code could be necessary. Therefore a detail level switch is required
Involved Elements	Show SFC

5.5 Summary

As a result of the experiments and the user feedback, we can state that the overall look and feel of an SFC is preserved whilst being layouted with the SFC Viewer. Imminent enhancements would be the introduction of *Goto Transitions* as well as a way to perform a more accurate layout for actions connected to steps, since the composite nodes introduce a different look to the SFC. For that a workshop with the developers of ELK and Sprotty is going to be organized in order to figure out ways to fix this particular problem.

The overall functionality demands for the SFC Viewer are of course subject to continuous improvement, which has to be refined in several additional feedback rounds with all the actors.

Chapter 6

Conclusion and Outlook

This thesis shows a possible way of displaying an automatically generated graphical Sequential Function Chart with web-technologies as a technological base. The motivation behind developing the SFC Viewer is the need for a future-oriented technology-base and the urge for constant renewal of the tool-chain for developing projects for the IEC 61131-3 standard.

First we took a look at the underlying state-of-the-art technologies. A short introduction to the IEC 61131-3 standard is followed by the introduction to the editor base Visual Studio Code and the idea of a Language Server Protocol both established by Microsoft. Additionally we outlined the theoretical background for drawing directed graphs in which we took a closer look at the layered algorithm of [14] and the Eclipse Layout Kernel.

In the concept and analysis part of the thesis we focused on a UML use case analysis as presented in [9]. In this analysis we defined the system context, the underlying use cases and ultimately the requirements of the to be developed system. The following main use cases were identified in the process: *Show SFC*, *Graphical Goto Definition*, *Goto Graphical Representation* and *Export SFC*. With this required knowledge a basic concept and the system architecture were defined on which we based the project.

After the ground for the implementation of the SFC Viewer was set, we selected a suitable drawing framework for the task. The definition of a Intermediate Representation Format and its conversion to a ELK readable format were followed by the introduction to the architecture of the chosen drawing framework - Sprotty. The mapping of the data structures between the different architectural layers was defined, and the realization of the first prototype was finally presented.

This prototype was finally evaluated against several SFC examples, which were compared to the look and feel of the former IEC editor. The prototype is capable of displaying several test SFCs, while preserving the overall look and feel. Also the identified use cases *Graphical Goto Definition*, *Goto Graphical Representation* and *Export SFC* were implemented and are functioning as intended. As a last step, feedback of users was taken into account which lead to the creation of the additional use cases *Find Element in SFC* and *Switch Detail Level* and their respective requirements.

Lastly, the next steps for the development of the SFC Viewer have to be outlined. Obviously, the already identified additional use cases and requirements have to be implemented. As a next step, further testing of all defined actors should be performed in order to extend the requirements to ultimately get closer to a usable product.

Furthermore, a big future development of the viewer is the extension to a full-fledged editor, which can be used either in a standalone way without the textual representation, or as a way of simple graphical editing with integration of the textual code on the fly. This project will need a whole new analysis of the system as well as a review of the in-place extension of the LSP to graphical languages, in order to function in both ways.

As a closing argument, it can be stated, that the overall functionality developed in this thesis, is already a good starting point for future developments towards the ultimate goal of bringing the SFC Viewer to be a productive part of the IEC 61131-3 development tool-chain.

List of Figures

2.1	Structure of an IEC project. Based on [5]	4
2.2	Example SFC: Top level control of an elevator cabin	6
2.3	Example SFC: Top level control of an elevator ride	7
2.4	Client-Server communication via language server protocol. Based on [6]	8
2.5	Drawing of a hierarchical acyclic digraph in upward direction. Based on [1, p. 22]	9
2.6	(a) Given digraph. (b) Assignment of vertices to layers (Step I). (c) Crossing reduction (Step II). (d) Placement of vertices and bends (Steps III and IV). Based on [1, p. 24]	10
2.7	ELK architecture. Based on [2]	11
3.1	System context for the SFC Viewer.	13
3.2	Use Case Model for the SFC Viewer	14
3.3	Refined Use Case Model for the SFC Viewer	16
3.4	Standard Sequence of use case <i>Show SFC</i>	17
3.5	Refined steps	18
3.6	Completed sequence of use case <i>Show SFC</i>	19
3.7	Layer model of the SFC viewer	22
3.8	Top level architecture of the SFC viewer	23
3.9	Architecture of the Logic/Data Layer	24
4.1	Use of composite node for changing layout direction	30
4.2	Split of transitions	30
4.3	Sprotty client architecture. Based on [12]	31
4.4	Class diagram of an ELK object	34
4.5	Extension architecture with sprotty	35
4.6	Prototype of the SFC Viewer	36
5.1	Example SFC: Top level control of an elevator cabin	38
5.2	Example SFC: Top level control of an elevator ride	39
5.3	Example: Large SFC with many transitions back to <i>Init</i>	40
5.4	Goto Transition in old SFC editor	41
5.5	Extended Use Case Model for the SFC Viewer	42

Listings

4.1	Top-level structure of IRF	27
4.2	Transition structure	28
4.3	Step structure	28
4.4	Action structure	29
4.5	IRF of Single-Step SFC	32
4.6	ELK-JSON object of Single-Step SFC	33
4.7	ElkGraph to SGraph conversion methods	34

Acronyms

API Application Programming Interface

DOM Document Object Model

DSL Domain-Specific Language

ELK Eclipse Layout Kernel

FBD Function Block Diagram

IDE Integrated Development Environment

IEC IEC 61131-3

IL Instruction List

IRF Intermediate Representation Format

JSON JavaScript Object Notation

LD Ladder Diagram

LSP Language Server Protocol

MVC Model View Controller

PLC Programmable Logic Controller

POU Program Organization Unit

RPC Remote Procedure Call

SEdge Sprotty Edge

SFC Sequential Function Chart

SGraph Sprotty Graph

SModel Sprotty Model

SNode Sprotty Node

ST Structured Text

SVG Scalable Vector Graphics

UML Unified Modeling Language

URI Uniform Resource Identifier

VSCode Visual Studio Code

Bibliography

- [1] G. Di Battista et al.
Algorithms for Drawing Graphs: an Annotated Bibliography.
Computational Geometry, October 1994
- [2] Eclipse Foundation.
<https://www.eclipse.org/elk/documentation.html>.
as of 17.05.2021.
- [3] Eclipse Foundation.
<https://www.eclipse.org/elk/documentation/tooldevelopers/graphdatastructure/jsonformat.html>.
as of 17.05.2021.
- [4] Eclipse Foundation.
<https://github.com/eclipse/sprotty/wiki/Architectural-Overview>.
as of 17.05.2021.
- [5] International Electrotechnical Commission.
Programmable Controllers, Part 3: Programming Languages.
2nd Edition, International Standard IEC 61131-3, 2003.
- [6] Microsoft.
<https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/>.
as of 17.05.2021.
- [7] Microsoft.
<https://www.typescriptlang.org/docs/>.
as of 17.05.2021.
- [8] Microsoft.
<https://code.visualstudio.com/>.
as of 17.05.2021.
- [9] B. Oestereich and A. Scheithauer.
Analyse und Design mit der UML 2.5.
Oldenbourg Verlag München, 2013

- [10] OMG.
UML Diagram Interchange.
OMG 2006
- [11] A. Otto and K. Hellmann.
IEC 61131: A general overview and emerging trends.
IEEE Industrial Electronics Magazine, vol. 3, no. 4, pp. 27-31, Dec. 2009
- [12] N. Rentz.
Moving Transient Views from Eclipse to Web Technologies.
Master's Thesis, Department of Computer Science, Kiel University, Nov. 2018
- [13] R. Rodriguez-Echeverria et al.
Towards a Language Server Protocol Infrastructure for Graphical Modeling.
MODELS '18, October 14–19, 2018, Copenhagen, Denmark
- [14] K. Sugiyama et al.
Methods for Visual Understanding of Hierarchical System Structures.
International Conference on Cybernetics and Society, Denver, Oct. 1979.